2023011112-实验 3-数科 2403 郑瑾

第 2 题：把 echo “Hello, world”命令的输出赋值给 myname 变量并打印出它的值。写出完成此项工作的所有命令。

myname=$(echo "Hello, world")

echo $myname

```
zhengjin@cg:~$ myname=$(echo "hello,world")
zhengjin@cg:~$ echo $myname
hello,world
```

第 5 题：编写一个 Shell 脚本（例如 shellQ5.sh），运行该脚本时需要带一个参数，该参数是一个文件（即以./shellQ5.sh file 形式运行该脚本）。该 Shell 脚本需要实现的功能是判断该参数所表示的文件是不是一个普通文件，如果是，则打印出该文件所有者的名字和最后的修改日期。如果运行该 Shell 脚本时带有多个参数，则输出出错信息。

```
File  Edit  View  Terminal  Tabs  Help
#!/bin/bash

if [ $# -ne 1 ]; then
        echo "we only need 1 arg"
        exit 1
fi

if [ -f "$1" ]; then
        owner=$(stat -c '%U' "$1")

        mod_date=$(stat -c '%y' "$1" | cut -d',' -f1)

        echo "file_owner: $owner"
        echo "last exit time: $mod_date"
else
        echo "$1 not a simple file"
fi
~
```

```
zhengjin@cg:~$ vim shellq5.sh
zhengjin@cg:~$ /shellq5.sh file1
bash: /shellq5.sh: No such file or directory
zhengjin@cg:~$ chmod +x shellq5.sh
zhengjin@cg:~$ echo "this is a test file" > testfile.txt
zhengjin@cg:~$ ls -l testfile.txt
-rw-rw-r-- 1 zhengjin zhengjin 20 Mar 30 11:03 testfile.txt
zhengjin@cg:~$ ./shellq5.sh
we only need 1 arg
zhengjin@cg:~$ ./shellq5.sh testfile.txt f1.txt
we only need 1 arg
zhengjin@cg:~$ ./shellq5.sh testfile.txt
file_owner: zhengjin
last exit time: 2026-03-30 11:03:13.306132111 +0000
zhengjin@cg:~$ ./shellq5.sh /
/ not a simple file
zhengjin@cg:~$ ./shellq5.sh aaaaa.txt
aaaaa.txt not a simple file
```

第 6 题：编写一个 bash 脚本程序，用 for 循环实现将当前目录下的所有.c 文件移到指定的目录下，最后在显示器上列举指定目录下的文件和目录。

File Edit View Terminal Tabs Help

```
#!/bin/bash

if [ $# -ne 1 ]; then
        echo "usage:$0 target directory"
        exit 1
fi

dest_dir="$1"

if [ ! -d "$dest_dir" ];then
        echo "the directory $dest_dir is not exist"
        exit 1
fi

count=0
for file in *.c; do
        if [ -f "$file" ]; then
                mv "$file" "$dest_dir/"
                echo "already moved:$file"
                ((count++))
        fi
done
echo "already moved $count .c files"
echo -e "\ntarget directory $dest_dir content:"
ls -la "$dest_dir"
~
```

```
zhengjin@cg:~$ vim move_c_files.sh
zhengjin@cg:~$ mkdir c_files_backup
zhengjin@cg:~$ ./move_c_files.sh c_files_backup
bash: ./move_c_files.sh: Permission denied
zhengjin@cg:~$ chmod +x move_c_files.sh
zhengjin@cg:~$ ./move_c_files.sh c_files_backup
already moved:6-1.c
already moved:6-2.c
already moved:6-3.c
already moved:6-4.c
already moved:6-5.c
already moved:6-6.c
already moved:6-7.c
already moved 7 .c files

target directory c_files_backup content:
total 8
drwxrwxr-x 2 zhengjin zhengjin 4096 Mar 30 11:20 .
drwxr-xr-x 6 zhengjin zhengjin 4096 Mar 30 11:20 ..
-rw-rw-r-- 1 zhengjin zhengjin    0 Mar 29 07:18 6-1.c
-rw-rw-r-- 1 zhengjin zhengjin    0 Mar 29 07:18 6-2.c
-rw-rw-r-- 1 zhengjin zhengjin    0 Mar 29 07:18 6-3.c
-rw-rw-r-- 1 zhengjin zhengjin    0 Mar 29 07:18 6-4.c
-rw-rw-r-- 1 zhengjin zhengjin    0 Mar 29 07:18 6-5.c
-rw-rw-r-- 1 zhengjin zhengjin    0 Mar 29 07:18 6-6.c
-rw-rw-r-- 1 zhengjin zhengjin    0 Mar 29 07:18 6-7.c
zhengjin@cg:~$ ./move_c_files.sh aaaaa
the directory aaaaa is not exist
```

第 9 题：写一个 Shell 脚本，包含两个数字数组 array1 和 array2，分别初始化为{1,2,3,4,5}和{1,4,9,16,25}（请注意，Shell 中声明/定义数组是用圆括弧且赋值时元素间不用逗号）。脚本生成并显示一个新数组，其元素是这两个数组中对应元素的和，例如新数组中第一个元素是 2，第 2 个元素是 6。

```
zhengjin@cg:~$ vim shellq9.sh
zhengjin@cg:~$ ./shellq9.sh
./shellq9.sh: line 4: [5: command not found
the first array:1 2 3 4 5
the second array:1 4 9 16 25
the new array:2 6 12 20 30

the ditail:
new array[0]=1+1=2
new array[1]=2+4=6
new array[2]=3+9=12
new array[3]=4+16=20
new array[4]=5+25=30
```

```
File  Edit  View  Terminal  Tabs  Help
#!/bin/bash
a1=(1 2 3 4 5)
a2=(1 4 9 16 25)
if [${#a1[@]} -ne ${#a2[@]} ]; then
        echo "wrong:the two arrays have different lengths"
        exit 1
fi

declare -a new_array
for ((i=0;i<${#a1[@]};i++)); do
        new_array[$i]=$(( ${a1[$i]}+${a2[$i]} ))
done

echo "the first array:${a1[@]}"
echo "the second array:${a2[@]}"
echo "the new array:${new_array[@]}"

echo -e "\nthe ditail:"
for ((i=0;i<${#new_array[@]};i++)); do
        echo "new array[$i]=${a1[$i]}+${a2[$i]}=${new_array[$i]}"
done
~
```

第 10 题：

当前目录下的 data 文件所包含的内容如下：

```
echo -n "The time now is:"
date
echo "The users presently logged in are:"
who
```

那么，执行如下命令序列的结果是什么？试着解释该命令序列的含义。

cat <data >out; chmod +x ./out; ./out

```
zhengjin@cg:~$ vim data
zhengjin@cg:~$ cat <data >out; chmod +x ./out; ./out
the time now is:Mon Mar 30 11:45:48 UTC 2026
the users presently logged in are:
root     pts/0        2026-03-30 10:45 (:1.0)
root     pts/2        2026-03-12 07:53 (:1.0)
root     pts/3        2026-03-12 07:56 (:1.0)
```

实验体会

通过本次实验，我深入理解了 Shell 编程的基本概念和操作：

变量赋值与命令替换：掌握了两种将命令输出赋值给变量的方法（反引号和$()），理解了命令替换的机制。

脚本参数处理：学会了通过#获取参数数量，通过 1、$2 等获取具体参数值，并实现基本的参数校验。

条件判断：熟练使用 if 语句和文件测试操作符（如-f 判断是否为普通文件）。

循环结构：掌握了 for 循环的两种常见用法（遍历文件和数值循环），理解了通配符在文件遍历

中的应用。
数组操作：学习了 Shell 中数组的声明、初始化和遍历方法，理解了数组索引从 0 开始的特点。
文件权限与执行：理解了 chmod 命令的作用，掌握了如何使脚本文件可执行。
重定向操作：加深了对输入输出重定向（<、>）的理解，认识到可以通过重定向创建脚本文件。
Shell 脚本作为系统管理和自动化的重要工具，其语法简洁但功能强大。通过实际编写和调试，我对 Shell 编程的逻辑结构和执行流程有了更直观的认识，为后续的 Linux 系统管理和自动化任务打下了坚实基础。

实验三
该题的任务是要判定学生成绩是否及格。请根据如下所给出的语句，填写未完成的部分，通过运行该程序（脚本），输出成绩判定信息（即及格、不及格、成绩录入错误（例如 148 分））；

for score in 35 90 55 60 100 83 148 75

...

```
zhengjin@cg:~$ ./grade_check.sh
score:35 - bujige
score:90 - jige
score:55 - bujige
score:60 - jige
score:100 - jige
score:83 - jige
score:148 make wrong
score:75 - jige
```

```
File  Edit  View  Terminal  Tabs  Help
#!/bin/bash
for score in 35 90 55 60 100 83 148 75
do
        if [ $score -lt 0 ] || [ $score -gt 100 ]; then
                echo "score:$score make wrong"
        elif [ $score -ge 60 ]; then
                echo "score:$score - jige"
        else
                echo "score:$score - bujige"
        fi
done
```

实验体会
通过完成这个成绩判定实验，我获得了以下体会：
1. Shell 脚本的条件判断语法
学习了使用 if [condition]语法进行条件判断
掌握了比较操作符：-lt（小于）、-gt（大于）、-ge（大于等于）等
理解了逻辑运算符的使用：||（或）、&&（与）
2. 循环结构应用

实践了使用 for 循环遍历数组元素
学会了如何在循环中使用 continue 跳过某些处理
3. 数据处理与验证
实现了对输入数据的有效性验证（检查分数是否在 0-100 范围内）
掌握了错误处理机制，能够识别并处理无效数据
4. 统计功能实现
学会了在循环过程中进行数据统计
掌握了百分比计算和数据显示格式化
5. 代码可扩展性
通过将常量（如及格分数线）定义为变量，提高了代码的可维护性
添加了等级划分和详细建议，使程序更加人性化
6. 实际应用价值
这个脚本可以实际应用于教学管理系统中的成绩判定
通过简单修改，可以适应不同的评分标准和阈值
7. 遇到的问题和解决方案
问题：在判断成绩是否有效时，需要注意边界条件（0 和 100 都是有效分数）
解决方案：使用 -ge 和 -le 或明确的范围判断 [$score -ge 0] && [$score -le 100]
8. 进一步改进的想法
可以添加从文件读取成绩的功能
可以添加成绩排序和排名功能
可以生成更详细的统计报告，包括平均分、最高分、最低分等
这个实验让我对 Shell 脚本编程有了更深入的理解，特别是条件判断和循环结构在实际问题中的应用。通过将理论知识与实际问题相结合，我不仅掌握了语法，还学会了如何设计和实现一个实用的程序。

实验 4

（1） 利用 while 语句，写出求 1-100 以内整数的累加和的语句，即 1+2+3+…+99+100

（2） 将该语句改为 for 语句，实现同样的功能

While 版本：

```
#!/bin/bash
i=1
sum=0
while [ $i -le 100 ]
do
        sum=$((sum+i))
        i=$((i+1))
done
echo "sum of 1 to 100 is $sum"
~
```

```
zhengjin@cg:~$ vim sum.sh
zhengjin@cg:~$ chmod +x sum.sh
zhengjin@cg:~$ ./sum.sh
sum of 1 to 100 is 5050
```

```
File  Edit  View  Terminal  Tabs  Help
#!/bin/bash
i=1
sum=0
for ((i=1;i<=100;i++))
do
        sum=$((sum+i))
done
echo "sum of 1 to 100 is $sum"
```

For 版本：

```
zhengjin@cg:~$ vim sum.sh
zhengjin@cg:~$ ./sum.sh
sum of 1 to 100 is 5050
```

实验体会

通过完成 1-100 整数累加和的计算，我有以下体会：

1. Shell 循环语法的掌握

while 循环：

语法：while [condition]; do ... done

适合不确定循环次数的情况

需要手动控制循环变量更新

示例中展示了基本计数器和条件判断

for 循环：

三种主要形式：

类 C 风格：for ((i=1; i<=100; i++))

序列形式：for i in $(seq 1 100)

范围形式：for i in {1..100}

适合已知循环次数的情况

语法更加简洁

2. 算术运算的多样性

使用了$((expression))进行算术运算

掌握了变量的自增操作：i=$((i + 1))或 ((i++))

理解了 Shell 中数值运算的基本规则

3. 语法细节的重要性

条件判断中方括号与表达式之间必须有空格

变量引用时使用$符号

字符串与数值的区别处理

实验 5

```
File  Edit  View  Terminal  Tabs  Help
#!/bin/bash
if [ $# -eq 0 ]; then
        echo "Your input can't be recognized."
        exit 1
fi

case $1 in
        "apple")
                echo "apple is red"
                ;;
        "banana")
                echo "banana is yellow"
                ;;
        "cat")
                echo "cat is cute"
                ;;
        "dog")
                echo "dog is loyal"
                ;;
        *)
                echo "Your input can't be recognized."
                ;;
esac
```

```
zhengjin@cg:~$ vim case.sh
zhengjin@cg:~$ chmod +x case.sh
zhengjin@cg:~$ ./case.sh apple
apple is red
zhengjin@cg:~$ ./case.sh banana
banana is yellow
zhengjin@cg:~$ ./case.sh cat
cat is cute
zhengjin@cg:~$ ./case.sh dog
dog is loyal
zhengjin@cg:~$ ./case.sh hello
Your input can't be recognized.
zhengjin@cg:~$ ./case.sh
Your input can't be recognized.
```